Modélisation de la probabilité de défaut

Documentation technique et fonctionnelle du code

Parcours MAFi

11 octobre 2026

Table des matières

1 Objet du document **3**

2 Installation et exécution **3**
2.1 Contenu du livrable 3
2.2 Exécuter le notebook 3
2.3 Organisation du notebook 4

3 Architecture **4**
3.1 Flux de données 4
3.2 Modules 5

4 Principes de conception **5**
4.1 Programmation fonctionnelle 5
4.2 Prévention des fuites d'information 6
4.3 Traçabilité 6

5 Description fonctionnelle des étapes **6**
5.1 Étape 1 : qualité des données 6
5.2 Étape 2 : valeurs aberrantes 6
5.3 Étape 3 : variables dérivées 6
5.4 Étape 4 : discrétisation, WoE et IV 7
5.5 Étape 5 : sélection des variables 7
5.6 Étapes 6 à 8 : modélisation 7

6 Algorithmes détaillés **7**
6.1 Traitements déterministes de qualité (`data_quality.apply_treatments`) 7
6.2 Détection et plafonnement des valeurs aberrantes (`outliers`) 8
6.3 Variables dérivées (`features`) 8
6.4 Discrétisation d'une variable quantitative (`binning.fit_numeric_binning`) 8
6.5 Regroupement d'une variable qualitative (`binning.fit_categorical_binning`) . 8
6.6 WoE et IV (`binning.woe_table`) 8
6.7 Stabilité et sélection (`selection`) 9
6.8 Sélection pas à pas (`modeling.stepwise`) 9
6.9 Robustesse, score et PD (`modeling`) 9
6.10 XGBoost (`xgb_model`) 9

7 Référence des fonctions **9**
7.1 `src.data_loading` . . . 9
7.2 `src.data_quality` . . . 10
7.3 `src.sampling` . . . 11
7.4 `src.outliers` . . . 12
7.5 `src.features` . . . 13
7.6 `src.imputation` . . . 14
7.7 `src.binning` . . . 14
7.8 `src.pipeline` . . . 15
7.9 `src.selection` . . . 15
7.10 `src.modeling` . . . 16
7.11 `src.xgb_model` . . . 17
7.12 `src.reporting` . . . 17
7.13 `src.plots`, `src.plots_outliers`, `src.plots_features` . . . 18
7.14 Compléments de référence . . . 18
7.15 Fonctions de visualisation et figures produites . . . 19

8 Paramètres de configuration **20**

9 Dictionnaire des fichiers produits **20**

10 Invariants, contrôles et reproductibilité **21**

11 Tests (partie 9 du notebook) **22**

12 Ajouter une étape **22**

1 Objet du document

Ce document décrit le code Python du projet :

— du point de vue **fonctionnel** : ce que fait chaque étape et pourquoi ;

— du point de vue **technique** : l'organisation des modules, les conventions de programmation et la référence des fonctions publiques.

La méthodologie statistique et les résultats sont présentés dans le rapport (`latex/rapport/`). Il couvre l'ensemble du pipeline : qualité des données, découpage apprentissage/test, valeurs aberrantes, variables dérivées, discrétisation et WoE, sélection des variables, régression logistique, robustesse, PD et XGBoost.

2 Installation et exécution

2.1 Contenu du livrable

```
credit_risk.ipynb            notebook unique : tout le code, toutes les analyses, les
    tests
data/LoanDetails.csv         données sources (jamais modifiées)
requirements.txt             dépendances Python
resultats/donnees/           échantillons produits par le notebook
resultats/tableaux/          tableaux de résultats (CSV)
resultats/tableaux_latex/    les mêmes tableaux au format LaTeX
resultats/figures/           figures (PNG)
rapport.tex, rapport.pdf     rapport : méthodologie, résultats, analyse
documentation.tex, .pdf      présente documentation
```

2.2 Exécuter le notebook

```
python -m venv .venv
.venv\Scripts\activate                # Windows
pip install -r requirements.txt
jupyter lab credit_risk.ipynb         # puis Kernel > Restart Kernel and Run All Cells
```

Ou, sans interface :

```
jupyter nbconvert --to notebook --execute --inplace credit_risk.ipynb
```

Le notebook doit être lancé depuis le dossier du livrable : les chemins sont relatifs à ce dossier. L'exécution complète dure quelques minutes ; la recherche sur grille de XGBoost en représente l'essentiel. Elle régénère à l'identique le contenu de `resultats/`.

2.3 Organisation du notebook

Bloc	Contenu
0. Mise en place	Définit la commande `%%module src.<nom>`. Elle exécute une cellule comme un module Python autonome enregistré dans `sys.modules`, ce qui conserve le découpage en modules et les imports `from src.config import ...`.
A. Code du projet	Les 17 modules, dans l'ordre de leurs dépendances : `config`, `data_loading`, `data_quality`, `reporting`, `binning`, `selection`, `sampling`, `outliers`, `features`, `imputation`, `modeling`, `xgb_model`, `plots`, `plots_outliers`, `plots_features`, `plots_model`, `pipeline`.
Parties 1 à 8	Analyses dans l'ordre du sujet (1.1 à 1.5.3). Chaque résultat est suivi de son interprétation.
Partie 9	Tests unitaires : chaque cellule `%%testfile` contient un fichier de tests, exécuté par `pytest` sur les modules du bloc A (59 tests).

Correspondance avec un dépôt Python classique. Le module `src.binning` du notebook correspond au fichier `src/binning.py` d'un projet structuré, et ainsi de suite. Les références de fonctions de ce document emploient la notation `src.module`.

Compilation des documents LaTeX. Une fois le notebook exécuté, depuis le dossier du livrable :

```
latexmk -pdf rapport.tex
latexmk -pdf documentation.tex
```

Sans `latexmk`, lancer `pdflatex rapport.tex` deux fois, pour la table des matières et les références croisées.

3 Architecture

3.1 Flux de données

Partie du notebook	Entrée	Sortie
1. Qualité des données	`data/LoanDetails.csv`	`loans_after_data_quality.csv` (9 827 prêts), puis `train_dq.csv` et `test_dq.csv`
2. Valeurs aberrantes	`train_dq.csv`, `test_dq.csv`	`train_outliers.csv`, `test_outliers.csv`
3. Variables dérivées	`train_outliers.csv` (montants plafonnés) et `train_dq.csv` (montants d'origine, pour les ratios) ; idem pour le test	`train_features.csv`, `test_features.csv`
4. Discrétisation, WoE	`train_features.csv`, `test_features.csv` (via `pipeline.prepare`)	tableaux et figures (aucun fichier de données : règles réapprises à la demande)
5. Sélection	idem	`variables_candidates.json`
6. Régression logistique	idem + `variables_candidates.json`	`variables_final.json`
7. Robustesse, PD	idem + `variables_final.json`	`pd_test_logistique.csv`
8. XGBoost	idem + `variables_final.json`	`pd_test_comparaison.csv`

Les fichiers de `resultats/donnees/` contiennent la clé `Loan ID` (non utilisée comme variable), les variables explicatives conservées et la cible `default` (1 = défaut).

3.2 Modules

Module	Responsabilité
`config`	Constantes : chemins, graine, rôles des variables, seuils du sujet, paramètres des traitements.
`data_loading`	Lecture du CSV brut, typage, codage de la cible.
`data_quality`	Analyses de qualité (Q1–Q4) et traitements déterministes.
`sampling`	Découpage apprentissage / test stratifié et reproductible.
`outliers`	Détection (3 écarts-types, IQR), comparaison, stratégies et plafonnement (Q5–Q6).
`features`	Variables dérivées DTI, CUR, ACR, ESR, IES : définitions économiques, divisions sûres, contrôles (1.2.1).
`imputation`	Imputations apprises sur l'apprentissage : hot-deck du montant, mode des faillites.
`binning`	Discrétisation en quantiles, Khiš, T de Tschuprow, V de Cramér, regroupements, WoE, IV (1.2.2–1.2.4).
`pipeline`	Préparation commune des notebooks 04–08 : imputation, discrétisation, WoE ; listes de variables.
`selection`	Bootstrap de stabilité, PSI, pouvoir discriminant, corrélations, jeu final (1.3).
`modeling`	Régression logistique, métriques, stepwise, bootstrap des coefficients, score, PD, contributions (1.4, 1.5.1–1.5.2).
`xgb_model`	XGBoost : variables d'entrée, recherche sur grille, comparaison, écart d'AUC bootstrap (1.5.3).
`reporting`	Export des tableaux en CSV et en `tabular` LaTeX.
`plots`, `plots_outliers`, `plots_features`, `plots_model`	Figures matplotlib, au style commun.

4 Principes de conception

4.1 Programmation fonctionnelle

Le sujet exprime une préférence pour la programmation fonctionnelle. Le code applique les règles suivantes.

— **Fonctions pures.** Une fonction renvoie un nouvel objet et ne modifie jamais ses entrées. Les traitements utilisent `copy`, `assign`, `mask`, `clip` ou `replace`, jamais une affectation en place sur l'argument. Des tests le vérifient, par exemple `test_replace_loan_amount_sentinel` et `test_capping_is_fitted_on_train_only`.

— **Pas d'état global.** Aucun `np.random.seed` : la graine est passée explicitement (`random_state=RANDOM_STATE`).

— **Apprentissage et application séparés.** Une fonction `fit_*` calcule des paramètres à partir de l'apprentissage et les renvoie sous forme immuable : `NamedTuple Bounds`, dictionnaire de bornes. Une fonction `apply_*` applique ces paramètres à n'importe quel échantillon. Il n'y a pas de classe à état de type `fit/transform`.

— **Composition.** Les traitements déterministes forment une liste d'étapes `(nom, fonction)` composée par `functools.reduce` (`apply_treatments`). Les méthodes de détection (`DETECTORS`), les stratégies de traitement (`strategies`) et les règles de cohérence (`consistency_rules`) sont des fonctions passées en paramètres : des fonctions d'ordre supérieur.

— **Effets de bord isolés.** Lecture et écriture de fichiers et production des figures sont confinées à `data_loading`, `reporting`, `plots.save` et aux notebooks.

4.2 Prévention des fuites d'information

Seules des corrections *déterministes* sont appliquées avant le découpage apprentissage/test : remplacement d'une valeur sentinelle, correction d'échelle, harmonisation de libellés, dédoublonnage. Elles ne dépendent d'aucune statistique de l'échantillon. Tout paramètre estimé (bornes de plafonnement, imputation, classes, WoE, sélection) est appris par une fonction `fit_*` sur l'apprentissage, puis appliqué au test.

4.3 Traçabilité

Chaque nombre du rapport provient d'un tableau généré par `save_table` ou `save_latex_table`, ou d'une sortie de notebook exécuté. Les décisions qualitatives sont codées comme données dans le code (`VARIABLE_DECISIONS`, `OUTLIER_DECISIONS`) et jointes aux effectifs calculés dans les tableaux de synthèse.

5 Description fonctionnelle des étapes

5.1 Étape 1 : qualité des données

1. **Chargement** en texte brut, pour distinguer les codages du manquant (vide, `NA`, `n/a`), puis typage.
2. **Inventaire** des variables (rôle, nature), codage et distribution de la cible.
3. **Identifiants et faible variabilité** : unicité, relation entre identifiants, part de la modalité dominante (seuil de 95 %), test du χ^2.
4. **Valeurs manquantes** : comptage par codage, nature (non applicable, réel, ambigu), taux de défaut avec et sans valeur, co-occurrence.
5. **Cohérence** : doublons, règles de plage et de cohérence entre variables, libellés équivalents.
6. **Traitements déterministes**, journalisés.
7. **Découpage** apprentissage / test et export.

5.2 Étape 2 : valeurs aberrantes

1. **Description** des distributions : forme, histogrammes, boîtes à moustaches.
2. **Détection** par les deux méthodes du sujet, sur l'apprentissage.
3. **Comparaison** : recouvrement, masquage, dégénérescence de l'IQR.
4. **Origine** : risque des extrêmes, extrêmes en ordre de grandeur, cohérence avec le revenu.
5. **Comparaison des stratégies** : suppression, imputation, plafonnement, transformation.
6. **Traitement retenu** : plafonnement au P99 appris sur l'apprentissage, appliqué aux deux échantillons.

5.3 Étape 3 : variables dérivées

1. **Définitions** : formule, signification économique et effet attendu de chaque variable (`FEATURE_DEFINITIONS`).
2. **Conversion** de l'ancienneté d'emploi en années (`< 1 year` → 0, `10+ years` → 10).
3. **Choix des montants** : les ratios sont calculés sur les montants non plafonnés ; la déformation qu'introduirait le plafonnement est mesurée.
4. **Divisions par zéro** : dénominateur nul ou manquant → valeur non définie ; les plafonds erronés sont traités comme inconnus ; les causes sont comptées.

5. **Construction** : apprentissage des paramètres sur l'apprentissage, application aux deux échantillons, plafonnement de l'IES.
6. **Contrôles** : distributions, outliers, effet observé contre effet attendu, redondances.

5.4 Étape 4 : discrétisation, WoE et IV

1. **Imputation** du montant (tirage dans la distribution de l'apprentissage) et des faillites (mode).
2. **Variables quantitatives** : pour $k = 2, \ldots, 10$, bornes aux quantiles. Les classes sous 5 % (observations, défauts ou encours) sont fusionnées avec leur voisine de taux le plus proche. On retient le k de T de Tschuprow maximal.
3. **Variables qualitatives** : modalités ordonnées (par taux de défaut, ou ordre naturel), fusion des petites modalités, puis fusion des voisines non significativement différentes.
4. **WoE et IV** par classe, appris sur l'apprentissage et appliqués au test.

5.5 Étape 5 : sélection des variables

1. **Quatre bootstraps** de l'apprentissage : taux de défaut, WoE et IV par classe, matrice des PSI, indicateurs de cohérence, k optimal par échantillon.
2. **V de Cramér et IV** avec la cible, p-value du χ^2.
3. **Paires** de V $> 0{,}4$ entre variables ; redondance si la corrélation des WoE dépasse 0,4.
4. **Décision** par variable (`final_selection`), puis enregistrement des candidates.

5.6 Étapes 6 à 8 : modélisation

1. **Représentativité** apprentissage/test : PSI et test d'homogénéité.
2. **Régression logistique** sur les candidates codées en WoE, puis **stepwise** (forward, backward, both ; AIC, BIC). Le modèle final est celui de BIC minimal parmi les modèles sans variable non significative ni coefficient négatif.
3. **100 bootstraps** du modèle final, **score et PD** calculés explicitement, calibration.
4. **XGBoost** : recherche sur grille en validation croisée, comparaison sur le test, intervalle bootstrap de l'écart d'AUC.

6 Algorithmes détaillés

Cette section décrit pas à pas les algorithmes non triviaux du code. Les notations sont celles du rapport : $y_i \in \{0, 1\}$ est la cible, n l'effectif de l'apprentissage, k un nombre de quantiles et r un nombre de classes.

6.1 Traitements déterministes de qualité (`data_quality.apply_treatments`)

1. Construire la liste ordonnée des étapes (nom, f) : sentinelle $\to$ NaN ; Credit Score > 850 divisé par 10 ; harmonisation des libellés ; dédoublonnage.
2. Composer les étapes par `functools.reduce`. L'état transmis est le couple (table, journal) ; chaque étape produit une nouvelle table et ajoute une ligne au journal (lignes avant/après, cellules modifiées).
3. Dédoublonnage : supprimer les doublons exacts, trier les lignes par nombre de valeurs manquantes croissant (tri stable), puis garder la première ligne de chaque *Loan ID*.

6.2 Détection et plafonnement des valeurs aberrantes (`outliers`)

1. Un *détecteur* est une fonction Série → `Bounds`(inf, sup) : $\bar{x} \pm 3s$ (`sigma_bounds`) ou $[Q_1 - 1{,}5\,\mathrm{IQR}, Q_3 + 1{,}5\,\mathrm{IQR}]$ (`iqr_bounds`). Les manquants sont ignorés.
2. `fit_capping` : pour chaque montant à plafonner, apprendre sur l'apprentissage la borne `Bounds`$(-\infty,\ q_{0,99})$.
3. `apply_capping` : renvoyer une nouvelle table où chaque montant vaut $\min(x, q_{0,99})$. Les mêmes bornes servent pour l'apprentissage et le test.

6.3 Variables dérivées (`features`)

1. `fit_feature_params` : apprendre la borne d'erreur des plafonds (clôture de Tukey $k = 3$ sur log) et le P99 de l'IES sur l'apprentissage.
2. `compute_ratios` : calculer DTI, CUR, ACR, ESR et IES ligne à ligne, sur les montants non plafonnés. Toute division passe par `safe_divide`, qui renvoie NaN si le dénominateur est nul ou manquant. Un plafond au-delà de la borne d'erreur est traité comme inconnu.
3. `add_features` : vérifier l'alignement des tables, puis ajouter les ratios à la table plafonnée.

6.4 Discrétisation d'une variable quantitative (`binning.fit_numeric_binning`)

1. Pour chaque $k = 2, \ldots, 10$:
   (a) bornes = quantiles d'ordre j/k des valeurs renseignées, doublons et maximum retirés ;
   (b) atomes = intervalles $]a\,;\,b]$, plus l'atome spécial ń manquant ż s'il existe ;
   (c) calculer χ^2, χ^2/n et T sur les atomes ;
   (d) **contrainte des 5 %** (`enforce_min_share`) : tant qu'une classe a moins de 5 % des observations, des défauts ou de l'encours, et qu'il reste plus de 2 classes, fusionner la classe la plus faible avec sa voisine de taux de défaut le plus proche. La classe spéciale rejoint la classe ordinaire de taux le plus proche ;
   (e) recalculer T sur les classes obtenues.
2. Retenir le k de T maximal après regroupement ; à égalité, celui qui a le moins de classes.
3. Renvoyer une règle immuable `BinningSpec` : bornes, correspondance atome → classe, ordre des classes, classe par défaut.

6.5 Regroupement d'une variable qualitative (`binning.fit_categorical_binning`)

1. Ordonner les modalités : par taux de défaut croissant (nominale) ou dans l'ordre naturel (ordinale). La modalité manquante est placée à part.
2. Appliquer la contrainte des 5 %, comme ci-dessus.
3. Fusion des profils semblables (`merge_similar`) : pour chaque paire de classes voisines, calculer la p-value du test du χ^2 2×2. Tant que la plus grande dépasse 5 % et qu'il reste plus de 2 classes, fusionner la paire correspondante.

6.6 WoE et IV (`binning.woe_table`)

1. Compter Bad_i et Good_i par classe. Ajouter 0,5 aux deux effectifs d'une classe dont l'un est nul.
2. $b_i = \mathrm{Bad}_i / \sum \mathrm{Bad}$, $g_i = \mathrm{Good}_i / \sum \mathrm{Good}$, $\mathrm{WoE}_i = \ln(b_i/g_i)$, $\mathrm{IV} = \sum_i (b_i - g_i)\,\mathrm{WoE}_i \geq 0$.
3. `fit_woe` mémorise les WoE de l'apprentissage ; `woe_transform` remplace chaque classe par son WoE, en apprentissage comme en test.

6.7 Stabilité et sélection (`selection`)

1. Tirer 4 vecteurs d'indices avec remise (graine 42). Pour chaque échantillon et chaque variable, calculer la table WoE avec les classes apprises.
2. PSI de chaque paire d'échantillons : $\sum_i (Q_i - P_i)\ln(Q_i/P_i)$, avec une proportion plancher de 10^{-4}.
3. Pour chaque paire de variables de V de Cramér $> 0{,}4$: calculer le V hors classes ń manquant ż et la corrélation des WoE. La paire est redondante si $|\rho_{\text{WoE}}| > 0{,}4$.
4. Décision (`final_selection`), dans l'ordre :
   — exclusion si IV $< 0{,}02$ et χ^2 non significatif ;
   — exclusion si PSI $\geq 0{,}25$;
   — exclusion de la variable de plus faible IV d'une paire redondante ;
   — sinon, candidate (ń faible ż si IV $< 0{,}02$).

6.8 Sélection pas à pas (`modeling.stepwise`)

1. Initialiser : ensemble vide (forward, both) ou toutes les variables (backward) ; critère $C =$ AIC ou BIC du modèle correspondant.
2. Énumérer les mouvements permis : ajout de chaque variable absente (forward, both), retrait de chaque variable présente (backward, both). Estimer le modèle de chaque mouvement.
3. Si le meilleur mouvement fait baisser C, l'appliquer et recommencer ; sinon arrêter.
4. Le modèle final est celui de BIC minimal parmi les modèles dont toutes les variables sont significatives et de coefficient positif.

6.9 Robustesse, score et PD (`modeling`)

1. `bootstrap_coefficients` : pour $b = 1, \ldots, 100$, tirer n indices avec remise et réestimer le modèle. Résumer par la moyenne, l'écart-type, le biais, les percentiles à 2,5 % et 97,5 %, et la part de coefficients de même signe que le modèle initial.
2. `logistic_score` : $S_i = \beta_0 + \sum_j \beta_j \text{WoE}_{ij}$; `pd_from_score` : $\text{PD}_i = 1/(1 + e^{-S_i})$.

6.10 XGBoost (`xgb_model`)

1. `fit_levels` et `raw_features` : construire des indicatrices pour les variables qualitatives (modalités apprises sur l'apprentissage) et garder les valeurs continues, avec leurs manquants.
2. `grid_search` : validation croisée stratifiée à 5 plis sur les 72 combinaisons de la grille, critère AUC, puis réestimation du meilleur modèle sur tout l'apprentissage.
3. `bootstrap_auc_difference` : 1 000 tirages avec remise du test ; pour chacun, écart d'AUC entre les deux modèles ; intervalle percentile à 95 %.

7 Référence des fonctions

7.1 `src.data_loading`

Fonction	Entrées	Sortie	Rôle
`load_raw`	chemin du CSV	DataFrame (texte)	Lit le fichier sans interpréter les manquants : les jetons `""`, `NA`, `n/a` sont conservés.
`to_typed`	DataFrame brut	DataFrame typé	Remplace les jetons de manquant par NaN et convertit les colonnes numériques ; échoue sur toute valeur non numérique inattendue.
`encode_target`	Série `Loan Status`	Série 0/1	Charged Off → 1, Fully Paid → 0 ; lève `ValueError` si une modalité inattendue ou manquante est présente.

7.2 `src.data_quality`

Analyses.

Fonction	Entrées	Sortie	Rôle
`variable_inventory`	DataFrame typé	tableau	Rôle, nature, type, nombre de modalités, manquants et exemples de chaque variable.
`target_summary`	cible codée, libellés	tableau	Effectifs et proportions des modalités de la cible.
`dataset_overview`	DataFrame, cible	tableau	Synthèse de la base : effectifs, natures, taux de défaut, manquants, doublons.
`numeric_description`	DataFrame, colonnes	tableau	Statistiques descriptives brutes des variables quantitatives.
`category_default_rates`	DataFrame, cible, colonne	tableau	Effectif, part et taux de défaut par modalité (manquant inclus).
`identifier_report`	DataFrame brut	tableau	Unicité des identifiants.
`identifier_cardinality`	DataFrame brut	tableau	Clients par prêt et prêts par client (maxima).
`dominant_modality_table`	DataFrame, cible, colonnes, seuil	tableau	Part de la modalité dominante, dépassement du seuil, taux de défaut et p-value du χ^2 (dominante contre autres).
`missing_token_counts`	DataFrame brut, colonnes	tableau	Nombre de cellules par jeton de manquant.
`missing_profile`	DataFrame typé, cible, colonnes	tableau	Taux de manquant, nature, taux de défaut si manquant ou renseigné, p-value.
`missing_value_table`	brut, typé, cible, colonnes	tableau	Jointure des deux tableaux précédents, triée par taux décroissant.

Fonction	Entrées	Sortie	Rôle
`missing_co_occurrence`	DataFrame, colonnes	matrice	Nombre de lignes où deux variables manquent simultanément.
`duplicate_summary`	DataFrame brut	tableau	Doublons exacts, Loan ID partagés, statuts contradictoires.
`near_duplicate_differences`	DataFrame brut	tableau	Colonnes qui diffèrent entre les lignes d'un même Loan ID.
`consistency_rules`	—	liste de règles	Règles `(nom, variables, fonction de masque)`.
`consistency_checks`	DataFrame, cible	tableau	Applique les règles : effectif, part et taux de défaut des anomalies.
`label_variants`	DataFrame, colonnes	tableau	Modalités ne différant que par la casse ou les séparateurs.
`quality_summary_table`	typé, traité, contrôles	tableau	Synthèse par variable : manquants avant et après, contrôles en anomalie, traitement, décision.

Traitements déterministes.

Fonction	Entrées	Sortie	Rôle
`replace_loan_amount_sentinel`	DataFrame	DataFrame	Montant 99 999 999 $\rightarrow$ NaN.
`rescale_credit_score`	DataFrame	DataFrame	Credit Score > 850 divisé par 10.
`harmonize_labels`	DataFrame, correspondances	DataFrame	Remplace les libellés équivalents.
`resolve_loan_id_duplicates`	DataFrame	DataFrame	Supprime les doublons exacts puis garde, par Loan ID, la ligne la moins incomplète.
`treatment_steps`	correspondances	liste d'étapes	Étapes `(nom, fonction)` dans leur ordre d'application.
`apply_treatments`	DataFrame typé, correspondances	(DataFrame, journal)	Compose les étapes avec `reduce` et journalise lignes et cellules modifiées.
`parse_years_in_job`	Série texte	Série numérique	`< 1 year` $\rightarrow$ 0, `10+ years` $\rightarrow$ 10.
`chi2_pvalue`	variable, cible	float	p-value du test du χ^2 d'indépendance (NaN si table dégénérée).

7.3 `src.sampling`

Fonction	Entrées	Sortie	Rôle
`stratified_split`	DataFrame, cible, taille du test, graine	`TrainTest-Split`	Découpage stratifié reproductible (scikit-learn), renvoyé dans un `NamedTuple (train, test)`.
`split_summary`	`TrainTestSplit`	tableau	Effectif, part et taux de défaut de la population et des deux échantillons.

7.4 `src.outliers`

Détection.

Fonction	Entrées	Sortie	Rôle
`sigma_bounds`	Série, $k = 3$	`Bounds`	$[\bar{x} - ks,\ \bar{x} + ks]$.
`iqr_bounds`	Série, $k = 1{,}5$	`Bounds`	$[Q_1 - k\,\mathrm{IQR},\ Q_3 + k\,\mathrm{IQR}]$.
`log_iqr_bounds`	Série, $k = 3$	`Bounds`	Clôtures de Tukey sur $\log x$ ($x > 0$) : extrêmes en ordre de grandeur.
`upper_percentile_ bounds`	Série, q	`Bounds`	Borne haute au quantile q, sans borne basse.
`is_outlier`	Série, `Bounds`	masque	Valeurs hors bornes ; un manquant n'est jamais signalé.
`fit_bounds`	DataFrame, colonnes, détecteur	dict	Applique un détecteur (fonction `Série` $\to$ `Bounds`) à chaque colonne.
`distribution_shape`	DataFrame, colonnes	tableau	Moyenne, médiane, écart-type, asymétrie, kurtosis, percentiles, extrêmes.
`detection_table`	DataFrame, cible, colonnes	tableau	Pour chaque méthode de `DETECTORS` : bornes, outliers bas et hauts, part, taux de défaut.
`method_overlap`	DataFrame, colonnes	tableau	Outliers communs ou propres à chaque méthode, Jaccard, IQR nul.
`suspicious_magnitudes`	DataFrame, colonnes	tableau	Zéros et valeurs hors clôtures logarithmiques.
`ratio_to_reference`	DataFrame, colonne, référence, masque	tableau	Ratio de contrôle métier pour les lignes signalées.

Traitement.

Fonction	Entrées	Sortie	Rôle
`delete`	Série, masque	Série	Suppression : valeurs signalées $\to$ NaN (pour le comptage).
`cap`	Série, `Bounds`	Série	Plafonnement (winsorisation).

Fonction	Entrées	Sortie	Rôle
`impute_median`	Série, masque	Série	Valeurs signalées remplacées par la médiane des autres.
`log_transform`	Série	Série	$\log(1+x)$.
`strategies`	Série d'apprentissage	dict nom → fonction	Stratégies comparées, paramétrées sur l'apprentissage.
`strategy_comparison`	DataFrame, cible, colonne	tableau	Observations conservées, valeurs modifiées, moments et taux de défaut après chaque stratégie.
`row_deletion_impact`	DataFrame, cible, colonnes	tableau	Lignes et défauts conservés si l'on supprime toute ligne signalée.
`quantile_bins_preserved`	Série avant, Série après	booléen	Vérifie que les classes de quantiles sont identiques.
`fit_capping`	apprentissage, colonnes, q	dict de `Bounds`	Apprend les bornes de plafonnement.
`apply_capping`	DataFrame, bornes	DataFrame	Applique les bornes apprises (nouvel objet).
`capping_report`	échantillons, bornes	tableau	Valeurs plafonnées par variable et par échantillon.
`decision_table`	détection	tableau	Effectifs par méthode joints aux décisions `OUTLIER_DECISIONS`.

7.5 `src.features`

Fonction	Entrées	Sortie	Rôle
`safe_divide`	numérateur, dénominateur	Série	Division ; NaN si le dénominateur est nul ou manquant (jamais `inf`).
`years_in_job`	DataFrame	Série	Ancienneté d'emploi en années (`< 1 year` → 0, `10+ years` → 10, `n/a` → NaN).
`plausible_limit`	DataFrame, `Bounds`	Série	Plafond de crédit, NaN au-delà de la borne d'erreur probable.
`compute_ratios`	DataFrame, `Bounds`	DataFrame	Calcule DTI, CUR, ACR, ESR, IES ligne à ligne, sans plafonnement.
`fit_feature_params`	apprentissage (montants d'origine)	`FeatureParams`	Apprend la borne d'erreur des plafonds (Tukey sur log) et le plafond P99 de l'IES.
`derive_features`	DataFrame, `FeatureParams`	DataFrame	Variables dérivées finales (IES plafonné).
`add_features`	base plafonnée, base d'origine, paramètres	DataFrame	Ajoute les variables dérivées à la base plafonnée ; vérifie l'alignement des deux tables.

Fonction	Entrées	Sortie	Rôle
`undefined_report`	DataFrame, paramètres	tableau	Valeurs non définies par variable et par cause.
`zero_denominator_profile`	DataFrame, cible	tableau	Prêts à plafond nul, avec ou sans encours, et taux de défaut.
`capping_distortion`	base d'origine, base plafonnée, paramètres	tableau	Écart entre ratios calculés sur montants plafonnés et non plafonnés.
`default_rate_by_quantile`	DataFrame, cible, colonne	tableau	Taux de défaut par classe de quantiles (descriptif).
`expected_vs_observed`	DataFrame, cible	tableau	Sens attendu, taux de défaut des quantiles extrêmes, Spearman, conformité, monotonie.
`definitions_table`	—	tableau	Formule, signification économique et effet attendu.

7.6 `src.imputation`

Fonction	Entrées	Sortie	Rôle
`fit_imputation`	apprentissage	`Imputation-Params`	Réservoir des montants observés et mode des faillites.
`hot_deck`	Série, réservoir, graine	Série	Remplace chaque manquant par une valeur tirée avec remise dans le réservoir.
`apply_imputation`	DataFrame, paramètres	DataFrame	Applique les deux imputations (nouvel objet).

7.7 `src.binning`

Fonction	Entrées	Sortie	Rôle
`contingency`	classes, cible	tableau	Table de contingence classes × cible.
`chi2_stat, chi2_between`	deux variables	float	Statistique du χ^2 (avec la cible ; entre deux variables).
`tschuprow_t, cramers_v`	χ^2, n, r, c	float	T de Tschuprow et V de Cramér.
`association`	classes, cible	dict	χ^2, χ^2/n, T, V, nombre de classes.
`quantile_edges`	Série, k	bornes	Bornes internes d'un découpage en k quantiles.
`numeric_atoms, categorical_atoms`	Série, bornes	Série	Atome de chaque observation (intervalle, modalité, ou libellé spécial si manquante).

Fonction	Entrées	Sortie	Rôle
`class_profile`	classes, cible, encours	tableau	Effectif, défauts, encours, parts et taux de défaut par classe.
`enforce_min_share`	atomes, cible, encours, ordre	correspondance	Fusionne les classes jusqu'à atteindre 5 % des observations, des défauts et de l'encours.
`merge_similar`	atomes, cible, correspondance, ordre	correspondance	Fusionne les classes voisines de risque non significativement différent.
`numeric_candidates`	Série, cible, encours	(tableau, règles)	Découpages $k = 2 \dots 10$: χ^2, T avant et après regroupement.
`fit_numeric_binning`	Série, cible, encours	(`BinningSpec`, tableau)	Règle de T maximal et tableau des candidats.
`fit_categorical_binning`	Série, cible, encours, type	`BinningSpec`	Regroupement d'une variable qualitative.
`apply_spec, apply_specs`	DataFrame, règle(s)	classes	Applique les règles apprises ; une modalité inconnue rejoint la classe par défaut.
`woe_table`	classes, cible	tableau	WoE, IV par classe, taux de défaut ; lissage de 0,5 si une classe n'a aucun défaut ou aucun non-défaut.
`fit_woe, woe_transform`	classes, cible	dict / DataFrame	Apprend les WoE et code les classes.
`iv_summary, iv_strength`	classes, cible	tableau	IV et interprétation (table 3 de l'annexe).

7.8 `src.pipeline`

Fonction	Entrées	Sortie	Rôle
`prepare`	—	`Prepared`	Charge les échantillons, impute, discrétise et code en WoE (règles apprises sur l'apprentissage).
`fit_binning`	apprentissage, cible	(règles, candidats)	Discrétisation de toutes les variables candidates.
`save_variables, load_variables`	nom, liste	—	Enregistre ou lit une liste de variables (JSON).

7.9 `src.selection`

Fonction	Entrées	Sortie	Rôle
`psi, psi_status`	distributions P, Q	float / texte	PSI (forme positive) et interprétation (table 4).
`bootstrap_indices`	n, nombre, graine	liste	Positions des échantillons bootstrap.

Fonction	Entrées	Sortie	Rôle
`bootstrap_woe`	classes, cible, règles, échantillons	tableau	Taux de défaut, WoE et IV par classe et échantillon.
`psi_matrix`	classes, échantillons	tableau	PSI de chaque variable pour chaque paire d'échantillons.
`woe_stability`	bootstrap, référence	tableau	IV (moyenne, écart-type, CV), changements de signe des WoE, concordance des rangs.
`optimal_k_by_sample`	données, cible, variables, échantillons	tableau	k retenu quand la discrétisation est réapprise sur chaque échantillon.
`discrimination_table`	classes, cible, IV	tableau	V de Cramér, IV, p-value du χ^2, rangs.
`pairwise_cramer_v`	classes	matrice	V de Cramér entre variables.
`correlated_pairs`	classes, WoE, règles, IV	tableau	Paires V $>$ 0,4, V hors manquants, corrélation des WoE, redondance.
`final_selection`	discrimination, PSI, stabilité, paires	tableau	Décision et justification par variable.

7.10 `src.modeling`

Fonction	Entrées	Sortie	Rôle
`fit_logit`	X, y	résultat statsmodels	Régression logistique avec constante.
`coefficient_table`	résultat	tableau	Coefficients, écarts-types, z, p-values, odds ratios et IC à 95 %.
`predict_pd`	résultat, X	Série	PD prédite.
`accuracy_ratio, log_likelihood, sample_metrics`	y, PD	float / dict	AR = 2 AUC − 1, log-vraisemblance, AUC, log-loss, Brier.
`model_metrics`	résultat, X, y	dict	Log-vraisemblance, AIC, BIC, Rš de McFadden, AUC, AR.
`confusion_table, classification_metrics`	y, PD, seuil	tableau / dict	Matrice de confusion, précision, rappel, spécificité, F1.
`train_test_comparison`	résultat, échantillons, seuil	tableau	Indicateurs sur l'apprentissage et le test, et leur écart.
`vif_table`	X	tableau	Facteurs d'inflation de la variance.
`stepwise`	X, y, direction, critère	(variables, chemin)	Sélection forward, backward ou bidirectionnelle par AIC ou BIC.
`compare_models`	modèles, échantillons	tableau	Indicateurs du sujet pour plusieurs listes de variables.

Fonction	Entrées	Sortie	Rôle
`bootstrap_coefficients`, `bootstrap_summary`	X, y, n	tableaux	Coefficients réestimés sur n bootstraps ; moyenne, écart-type, biais, IC, part de même signe.
`logistic_score`, `pd_from_score`	X, coefficients	Série	$S = \beta_0 + \sum \beta_j X_j$; $\text{PD} = 1/(1+e^{-S})$.
`distribution_summary`, `calibration_table`	PD (et y)	tableau	Statistiques et quantiles des PD ; PD moyenne contre taux observé par décile.
`contributions`	X (WoE), coefficients	tableau	Contribution relative des variables (annexe du sujet).

7.11 `src.xgb_model`

Fonction	Entrées	Sortie	Rôle
`fit_levels`, `raw_features`	apprentissage / DataFrame	dict / DataFrame	Modalités de référence ; variables traitées (indicatrices, manquants conservés).
`grid_search`	X, y, grille	`GridSearchCV`	Recherche sur grille en validation croisée stratifiée (AUC).
`cv_results_table`	recherche	tableau	Meilleures combinaisons, AUC de validation et d'apprentissage.
`comparison_table`	y, prédictions	tableau	Indicateurs de chaque modèle sur le même échantillon.
`bootstrap_auc_difference`	y, PD A, PD B	dict	Écart d'AUC, IC bootstrap à 95 %, part d'écarts négatifs.
`roc_points`, `feature_importance`, `pd_agreement`	—	tableau / dict	Courbe ROC, importance (gain), concordance des PD.

7.12 `src.reporting`

Fonction	Entrées	Sortie	Rôle
`save_table`	tableau, nom	chemin	CSV UTF-8 dans `resultats/tableaux/`.
`percent`, `number`, `pvalue`	nombre	texte LaTeX	Formateurs : pourcentage et nombre à la française, p-value ($< 0{,}001$).
`header_label`	nom de colonne	texte	Libellé d'en-tête lisible.
`latex_escape`	texte	texte	Échappe les caractères spéciaux LaTeX.
`to_latex_tabular`	tableau, formateurs, colonnes	texte	Environnement `tabular` (booktabs).

Fonction	Entrées	Sortie	Rôle
`save_latex_table`	tableau, nom, formateurs	chemin	Écrit le `tabular` dans `resultats/tableaux_latex/`.

7.13 `src.plots`, `src.plots_outliers`, `src.plots_features`

Chaque fonction `plot_*` reçoit des tableaux déjà calculés et renvoie une `Figure` matplotlib, sans l'enregistrer. L'enregistrement est fait par `plots.save`, et `set_style` fixe le style commun : palette catégorielle à ordre fixe, grille discrète, texte en encre neutre. Les fonctions disponibles sont les suivantes.

— `plots` : distribution de la cible, taux de manquants, taux de défaut selon le manquant, modalité dominante, taux de défaut des contrôles, correction du Credit Score, taux de défaut par modalité.

— `plots_outliers` : histogrammes avec bornes, boîtes à moustaches, parts d'outliers par méthode, taux de défaut des outliers, effet du plafonnement.

— `plots_features` : taux de défaut par quintile, matrice de corrélation.

— `plots_model` : T de Tschuprow selon k, profils de WoE, IV, cartes de chaleur (PSI, V de Cramér, corrélations), chemins du stepwise, coefficients bootstrap, distribution et calibration des PD, courbes ROC, nuage de PD, contributions, importance XGBoost.

7.14 Compléments de référence

Les fonctions ci-dessous, plus utilitaires, complètent les tableaux précédents.

Fonction	Entrées	Sortie	Rôle
`data_quality.with_missing_label`	Série	Série	Remplace les manquants par la modalité explicite ń Manquant ż.
`data_quality.changed_cells`	deux DataFrame	entier	Nombre de cellules modifiées par une étape (journal des traitements).
`binning.interval_labels`	bornes	liste	Libellés `]a ; b]` des intervalles.
`binning.select_candidate`	tableau des candidats	entier	k de T maximal après regroupement (à égalité, le moins de classes).
`binning.information_value`	classes, cible	float	IV d'une variable discrétisée.
`pipeline.load_samples`	—	(train, test)	Lit `train_features.csv` et `test_features.csv`.
`sampling.representativity`	classes train, classes test	tableau	PSI apprentissage/test et p-value du test d'homogénéité du χ^2.
`sampling.default_rate_by_class`	classes, cibles, variable	tableau	Part et taux de défaut par classe dans les deux échantillons.
`selection.class_distribution`	classes	Série	Proportion des observations par classe (entrée du PSI).

Fonction	Entrées	Sortie	Rôle
selection.cramer_v_observed	classes, deux variables	float	V de Cramér hors classes contenant la modalité ń manquant ż.
reporting.auto_number	nombre	texte	Entier sans décimale, sinon jusqu'à 4 décimales.
xgb_model.base_model	graine	XGBClassifier	Modèle de base : objectif logistique, graine fixée, un seul fil d'exécution.
xgb_model.safe_name	nom	texte	Nom de colonne accepté par XGBoost.

7.15 Fonctions de visualisation et figures produites

Chaque figure de resultats/figures/ est produite par une fonction pure, qui renvoie une Figure sans l'enregistrer. Le notebook l'enregistre ensuite avec plots.save.

Fonction	Figure	Contenu
plots.plot_target_distribution	01_distribution_cible	Effectifs de la cible
plots.plot_category_default_rates	01_taux_defaut_qualitatives	Taux de défaut par modalité
plots.plot_dominant_modality	03_modalite_dominante	Part de la modalité dominante
plots.plot_missing_rates	04_taux_manquants	Taux de manquants et nature
plots.plot_missing_default_rates	04_taux_defaut_selon_manquant	Taux de défaut selon la présence de la valeur
plots.plot_check_default_rates	05_controles_taux_defaut	Taux de défaut des anomalies
plots.plot_credit_score_correction	05_credit_score_correction	Score avant et après correction
plots_outliers.plot_histograms_with_bounds	08_histogrammes_bornes, 10_histogrammes_variables_derivees	Histogrammes et bornes des deux méthodes
plots_outliers.plot_boxplots	08_boites_a_moustaches	Boîtes à moustaches
plots_outliers.plot_detection_counts	08_parts_outliers_par_methode	Part d'outliers par méthode
plots_outliers.plot_outlier_default_rates	08_taux_defaut_outliers	Taux de défaut des outliers
plots_outliers.plot_capping_effect	09_effet_plafonnement	Distributions avant et après plafonnement
plots_features.plot_default_rate_by_quantile	10_taux_defaut_quintiles	Taux de défaut par quintile
plots_features.plot_correlation_heatmap	10_correlations_variables_derivees	Corrélations de Spearman
plots_model.plot_tschuprow_by_k	11_tschuprow_par_k	T selon le nombre de quantiles
plots_model.plot_iv_ranking	11_iv_variables	Classement par IV
plots_model.plot_woe_profiles	11_woe_profils	WoE par classe
plots_model.plot_matrix	12_matrice_psi, 12_matrice_cramer_v, 12_correlation_woe	Cartes de chaleur annotées

Fonction	Figure	Contenu
plots_model.plot_stepwise_paths	13_stepwise_bic	Critère à chaque étape du stepwise
plots_model.plot_contributions	13_contributions	Contributions au score
plots_model.plot_bootstrap_coefficients	14_bootstrap_coefficients	Coefficients des 100 bootstraps
plots_model.plot_pd_distribution	14_distribution_pd, 15_distribution_pd_test	Histogrammes des PD
plots_model.plot_calibration	14_calibration_logistique, 15_calibration_test	PD moyenne et taux observé par décile
plots_model.plot_roc	15_roc_test	Courbes ROC
plots_model.plot_pd_scatter	15_pd_logistique_vs_xgboost	PD des deux modèles
plots_model.plot_importance	15_importance_xgboost	Importance XGBoost (gain)

8 Paramètres de configuration

Tous les paramètres sont centralisés dans `src.config`. Seules quelques constantes propres à un module (lissage du WoE, plancher du PSI, grille XGBoost) sont définies dans ce module.

Paramètre	Valeur	Rôle
RANDOM_STATE	42	Graine de tous les tirages : découpage, bootstraps, hot-deck, XGBoost
TEST_SIZE	0,20	Part de l'échantillon de test
LOAN_AMOUNT_SENTINEL	99 999 999	Valeur sentinelle du montant
CREDIT_SCORE_MAX, CREDIT_SCORE_SCALE_FACTOR	850, 10	Borne de l'échelle du score ; facteur de correction
DOMINANT_MODALITY_THRESHOLD	0,95	Seuil de faible variabilité (sujet 1.1, Q2)
HIGH_MISSING_THRESHOLD	0,10	Seuil de manquants à justifier (Q3)
SIGNIFICANCE_LEVEL	0,05	Niveau des tests du χ^2 (regroupements, sélection)
CAPPING_COLUMNS, CAPPING_QUANTILE	4 montants, 0,99	Plafonnement (Q6)
CORRELATION_THRESHOLD	0,4	Seuil de corrélation (sujet 1.3.3)
binning.MIN_SHARE	0,05	Part minimale d'observations, de défauts et d'encours par classe
binning.SMOOTHING	0,5	Correction de continuité du WoE
selection.PSI_EPSILON	10^{-4}	Proportion plancher d'une classe vide dans le PSI
selection.IV_MIN	0,02	Seuil ń non prédictive ż de l'IV
selection.PSI_STABLE, PSI_WATCH	0,10, 0,25	Seuils du PSI (table 4)
xgb_model.PARAM_GRID	72 combinaisons	Grille des hyperparamètres XGBoost
LABEL_HARMONIZATION	dictionnaire	Libellés équivalents de *Purpose* et *Home Ownership*

9 Dictionnaire des fichiers produits

Fichier (`resultats/donnees/`)	Notebook	Contenu
`loans_after_data_quality.csv`	01	9 827 prêts : *Loan ID*, 15 variables explicatives, cible `default`
`train_dq.csv`, `test_dq.csv`	01	Découpage 80/20 stratifié (7 861 / 1 966 prêts), montants non plafonnés
`train_outliers.csv`, `test_outliers.csv`	02	Mêmes prêts, 4 montants plafonnés au P99 de l'apprentissage
`train_features.csv`, `test_features.csv`	03	Ajout de DTI, CUR, ACR, ESR, IES (ratios sur montants non plafonnés)
`variables_candidates.json`	05	Liste des 14 variables candidates
`variables_final.json`	06	Liste des 8 variables du modèle final
`pd_test_logistique.csv`	07	Score et PD logistique de chaque prêt du test
`pd_test_comparaison.csv`	08	PD logistique et PD XGBoost de chaque prêt du test

Les tableaux de résultats sont écrits dans `resultats/tableaux/` (CSV) et `resultats/tableaux_latex/` (LaTeX). Le préfixe numérique de chaque fichier indique l'étape : 01–07 qualité, 08–09 outliers, 10 variables dérivées, 11 discrétisation, 12 sélection, 13 régression logistique, 14 robustesse et PD, 15 XGBoost.

10 Invariants, contrôles et reproductibilité

Contrôles qui lèvent une erreur. Le code échoue explicitement plutôt que de produire un résultat silencieusement faux :

- `to_typed` : valeur non numérique inattendue dans une colonne numérique ;
- `encode_target` : modalité de cible inconnue ou manquante ;
- `add_features` : tables d'entrée non alignées ligne à ligne ;
- `apply_spec` : observation sans classe ;
- `woe_transform` : classe sans WoE.

Invariants vérifiés dans les notebooks.

- unicité des *Loan ID* après dédoublonnage ;
- intersection vide entre apprentissage et test ;
- aucune valeur infinie dans les ratios ;
- toutes les observations du test reçoivent une classe ;
- score explicite égal à la prédiction du modèle (10^{-16} près) ;
- $0 \leq \text{PD} \leq 1$.

Reproductibilité. Toute source d'aléa reçoit `RANDOM_STATE` en argument : découpage, 4 et 100 bootstraps, hot-deck, validation croisée et XGBoost (`n_jobs=1` pour le modèle). Réexécuter les notebooks 01 à 08 dans l'ordre reproduit à l'identique tous les tableaux et figures, ce qui a été vérifié par une réexécution complète.

11 Tests (partie 9 du notebook)

Fichier	Ce qui est vérifié
`test_data_quality.py`	Codage de la cible (et rejet des modalités inattendues), lecture des anciennetés, comptage des jetons de manquant, sentinelle, correction d'échelle, dédoublonnage (ligne la plus complète), harmonisation, détection des libellés, modalité dominante, χ^2 dégénéré, comptage des cellules modifiées.
`test_outliers.py`	Bornes conformes aux définitions du sujet, manquants jamais signalés, effet de masquage, plafonnement, suppression, imputation, apprentissage des bornes sur l'apprentissage seulement (et entrée non modifiée), IQR nul, préservation des classes de quantiles.
`test_features.py`	Formules du sujet, ACR $= 1 -$ CUR, divisions par zéro ou par un manquant $\rightarrow$ NaN (jamais `inf`), ancienneté `< 1 year`, plafond erroné, comptage des causes, plafonnement de l'IES, alignement des tables, comparaison attendu/observé.
`test_binning.py`	Formules de T et V, quantiles, WoE et IV (signe positif, lissage), seuils de l'IV, choix du T maximal, contrainte de 5 %, fusion de la classe manquante, regroupement qualitatif et modalité inconnue, hot-deck reproductible.
`test_selection_modeling.py`	PSI (positif, symétrique, classe vide), bootstrap reproductible, redondance parfaite, AIC et AR, score explicite = prédiction, stepwise (3 directions), matrice de confusion, bootstrap et contributions, écart d'AUC.
`test_sampling_reporting.py`	Découpage stratifié, reproductible, sans recouvrement ; formateurs et échappement LaTeX.

12 Ajouter une étape

1. Écrire les fonctions dans une cellule `%%module src.<nom>` du bloc A : fonctions pures, avec `fit_*` et `apply_*` si des paramètres sont estimés.
2. Ajouter une cellule `%%testfile test_<nom>.py` dans la partie 9.
3. Ajouter une partie d'analyse qui lit les fichiers de l'étape précédente dans `resultats/donnees/` et exporte ses tableaux et figures.
4. Compléter le rapport et la présente documentation.